

MANUAL DE ADMINISTRACIÓN CORTA DE TYPE-PEPTIDE

1. Alcance

Este manual resume las tareas básicas para:

- Desplegar Type-Peptide con Docker.
- Verificar el estado de los servicios.
- Consultar logs y hacer cambios mínimos en la configuración.

2. Arquitectura de despliegue

Type-Peptide se ejecuta como dos contenedores coordinados por `docker-compose.yml`:

1. Servicio api

- Imagen Node.js construida desde `./api`.
- Expone internamente el puerto **8052**.
- Ofrece endpoints:
 - `GET /health`
 - `GET /api/info`
- No expone puerto directamente al host; se comunica por red Docker.

2. Servicio frontend

- Imagen `nginx:alpine`.
- Sirve el contenido estático de `./frontend` en el puerto **80**.
- Hace de proxy inverso hacia el servicio `api` en `/api/`.

Ambos servicios se conectan a la red externa `inverpep_default`.

3. Archivos relevantes

- `docker-compose.yml`
Orquesta los servicios `api` y `frontend`, define env vars y healthchecks.
- `nginx.conf`
Configuración del servidor Nginx:
 - `root /usr/share/nginx/html;`
 - Proxy `/api/` → `http://api:8052`
 - Health `GET /health` → "ok\n"
 - Políticas de caché (no-cache para HTML/JS/CSS; caché prolongada para imágenes).
- `api/.env` (opcional en desarrollo)
Variables de entorno para la API. En producción es recomendable usar variables o secretos de Docker/Swarm.

- `api/sequences/BD_FINAL.fasta`
Archivo FASTA cuya existencia se comprueba en `GET /health` para monitoreo.
Type-Peptide no depende funcionalmente de este archivo para la visualización, pero el healthcheck lo reporta.

4. Variables de entorno principales

En `docker-compose.yml` (servicio `api`):

- `PORT=8052`
Puerto interno de la API.
- `FASTA_PATH=/app/sequences/BD_FINAL.fasta`
Ruta esperada del FASTA dentro del contenedor.
- `FORCE_FASTA_ONLY=false`
Reservado para usos futuros; permite indicar si se trabajará solo con FASTA.
- Variables de DB (`DB_*`) heredadas del proyecto base, **actualmente no utilizadas** por Type-Peptide, pero se mantienen por compatibilidad.

En `frontend`:

- `VIRTUAL_HOST`, `LETSENCRYPT_HOST`, `LETSENCRYPT_EMAIL`, `VIRTUAL_PORT`
Se utilizan cuando la app está detrás de un proxy automático (nginx-proxy + Let's Encrypt).

5. Despliegue inicial

1. Posicionarse en la carpeta donde está `docker-compose.yml` de Type-Peptide.
Construir la imagen de la API:
`docker compose build --no-cache api`
2. Levantar servicios en segundo plano:
`docker compose up -d`
3. Verificar contenedores:
`docker compose ps`

6. Healthchecks

6.1. Frontend

- Endpoint: `GET /health`

Respuesta esperada:

- `ok`

6.2. API

Dentro de la red Docker (o usando port-forward):

- `GET /health`

Devuelve JSON indicando si existe el archivo FASTA configurado:

```
{
  "status": "ok",
  "fastaPath": "/app/sequences/BD_FINAL.fasta",
  "fastaExists": true
}
```

- `GET /api/info`

Devuelve información básica de versión y entorno.

Estos endpoints se usan en el `healthcheck` de Docker para reinicio automático si algo falla.

7. Logs

Para revisar registros en tiempo real:

- Logs de API:
`docker compose logs -f api`
- Logs de frontend (Nginx):
`docker compose logs -f frontend`
- Para cortar la visualización de logs, use `Ctrl + C`.

8. Actualización de la aplicación

8.1. Cambios en el frontend (HTML/CSS/JS)

1. Modificar archivos en `frontend/` (`index.html`, `app.js`, `peptide-utils.js`, `styles.css`, etc.).

Reiniciar solo el servicio frontend:

`docker compose restart frontend`

2. Si los cambios no se reflejan por caché del navegador, hacer recarga forzada (`Ctrl+F5`).

Como el frontend se monta como volumen de solo lectura, los cambios en los archivos del host se reflejan directamente en el contenedor.

8.2. Cambios en la API

1. Actualizar código en `api/server.js` u otros archivos. Reconstruir la imagen:

`docker compose build api`

`docker compose up -d api`

9. Respaldo y restauración

- **Código y configuración:**
 - Mantener un repositorio (ej. Git) con la carpeta del proyecto.
- **FASTA** (si se usa para monitoreo o futuras funciones):
 - Respaldar `api/sequences/BD_FINAL.fasta`.
- **Nginx:**
 - Guardar copia de `nginx.conf`.

Para restaurar en un nuevo servidor:

1. Clonar o copiar el proyecto.
2. Restaurar `BD_FINAL.fasta` si aplica.
3. Ejecutar `docker compose build` y `docker compose up -d`.

10. Problemas frecuentes

- **El healthcheck de la API falla:**
 - Revisar `docker compose logs -f api`.
 - Verificar que `FASTA_PATH` apunte a un archivo existente (o aceptar que `fastaExists` sea `false` si no se requiere).
- **Error 502 / Bad Gateway:**
 - Nginx no puede comunicarse con `api`.
 - Verificar que el servicio `api` está corriendo y que el `proxy_pass` en `nginx.conf` usa `http://api:8052`.

11. Autores y créditos

- **Autora principal:** Hinara Pastora Sánchez Mata.
- **Crédito institucional:** “Desarrollado por la línea de interacciones moleculares — Biología Funcional”.